

Unidad III: Complejidad Computacional

La clase NP

Clase 16 - Lógica para Ciencia de la Computación/Teoría de la Computación

Prof. Miguel Romero

La clase NP: motivación

¿Qué tienen en común los siguientes problemas?

$\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ fórmula proposicional satisfacible}\}.$

$\text{COMPOSITES} = \{\langle n \rangle \mid n = p \cdot q \text{ para enteros } p, q > 1\}.$

$\text{3-COL} = \{\langle G \rangle \mid G \text{ grafo no dirigido 3-coloreable}\}.$

La clase NP: motivación

Los problemas anteriores tienen la siguiente estructura:

- Dada una instancia, ¿existe una solución?
- Las soluciones tienen tamaño polinomial.
- Se puede **verificar eficientemente** si una solución dada es correcta.

La clase NP: definición

Definición:

Un lenguaje $L \subseteq \Sigma^*$ es **verificable en tiempo polinomial** si existe un **polinomio** $p: \mathbb{N} \rightarrow \mathbb{N}$ y un **MTD** M que toma tiempo polinomial tal que para toda palabra $w \in \Sigma^*$:

$$w \in L \iff \text{existe } x \in \Sigma^* \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } \langle w, x \rangle$$

Notación:

- x es un certificado para $w \in L$.
- M es un verificador para L .

Definición:

La clase **NP** es la clase de todos los lenguajes L que se pueden verificar en tiempo polinomial.

Ejemplos

$\langle \varphi \rangle \in \text{SAT} \iff \text{existe una valuación } \sigma \text{ tal que } \sigma(\varphi) = 1.$

El **certificado** para $\langle \varphi \rangle \in \text{SAT}$ es la valuación σ .

- Una codificación razonable de σ cumple $|\langle \sigma \rangle| \leq p(|\langle \varphi \rangle|)$, para un polinomio p .

El **verificador** es la siguiente MTD M :

Sobre entrada $\langle \varphi, x \rangle$:

1. Chequear que x codifica una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

M toma tiempo polinomial en $|\langle \varphi \rangle|$.

$\langle \varphi \rangle \in \text{SAT} \iff \text{existe } x \text{ tal que } |x| \leq p(|\langle \varphi \rangle|) \text{ y } M \text{ acepta } \langle \varphi, x \rangle.$

Concluimos que $\text{SAT} \in \text{NP}$.

Ejemplos

$\langle n \rangle \in \text{COMPOSITES} \iff \text{existe entero } 1 < p < n \text{ tal que } p \text{ divide } n.$

El **certificado** para $\langle n \rangle \in \text{COMPOSITES}$ es el entero p .

■ Tenemos que $|\langle p \rangle| \leq |\langle n \rangle|$.

El **verificador** es la siguiente MTD M :

Sobre entrada $\langle n, x \rangle$:

1. Chequear que x codifica un entero $1 < p < n$.
2. Si p divide n , **aceptar**. En caso contrario, **rechazar**.

M toma tiempo polinomial en $|\langle n \rangle|$.

$\langle n \rangle \in \text{COMPOSITES} \iff \text{existe } x \text{ tal que } |x| \leq |\langle n \rangle| \text{ y } M \text{ acepta } \langle n, x \rangle.$

Concluimos que $\text{COMPOSITES} \in \text{NP}$.

Ejemplos

$\langle G \rangle \in 3\text{-COL} \iff \text{existe un 3-coloreo } c \text{ para } G.$

El **certificado** para $\langle G \rangle \in 3\text{-COL}$ es el 3-coloreo c .

- Una codificación razonable de c cumple $|\langle c \rangle| \leq p(|\langle G \rangle|)$, para un polinomio p .

El **verificador** es la siguiente MTD M :

Sobre entrada $\langle G, x \rangle$:

1. Chequear que x codifica una función c de los nodos de G a $\{1, 2, 3\}$.
2. Si para toda arista (u, v) de G se cumple $c(u) \neq c(v)$, **aceptar**.
En caso contrario, **rechazar**.

M toma tiempo polinomial en $|\langle G \rangle|$.

$\langle G \rangle \in 3\text{-COL} \iff \text{existe } x \text{ tal que } |x| \leq p(|\langle G \rangle|) \text{ y } M \text{ acepta } \langle G, x \rangle.$

Concluimos que $3\text{-COL} \in \text{NP}$.

Ejemplos

Considere el siguiente problema:

$\text{TAUT} = \{ \langle \varphi \rangle \mid \varphi \text{ fórmula proposicional que es una tautología} \}.$

¿Está TAUT en NP?

Definición:

Una **máquina de Turing no-determinista** (MTN) es una tupla

$M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$, donde:

- Q es un conjunto finito de **estados**.
- Σ es el **alfabeto de entrada**.
- Γ es el **alfabeto de la cinta**, donde $\Sigma \subseteq \Gamma$, y $\vdash, B \in \Gamma \setminus \Sigma$.
- q_0 es el **estado inicial**.
- q_f es el **estado final**.
- $\Delta \subseteq (Q \setminus \{q_f\}) \times \Gamma \times Q \times \Gamma \times \{\triangleleft, \triangleright\}$ es la **relación de transición**.

Recordatorio: ejecuciones de una MTN

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

Definición:

- Una **ejecución** ρ de M es una secuencia de configuraciones

$$\rho = C_0, C_1, C_2, \dots \text{ (no necesariamente finita)}$$

tal que $C_i \xrightarrow{M} C_{i+1}$ para todo $i \geq 0$.

- Decimos que $\rho = C_0, C_1, C_2, \dots$ es una **ejecución** de M sobre w si:

- ρ es una ejecución de M .
- $C_0 = \vdash q_0 w$. (configuración inicial de M sobre w)
- Si $\rho = C_0, \dots, C_m$ es finita, entonces C_m es de detención.

Ahora es posible tener **varias** ejecuciones de M sobre w .

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

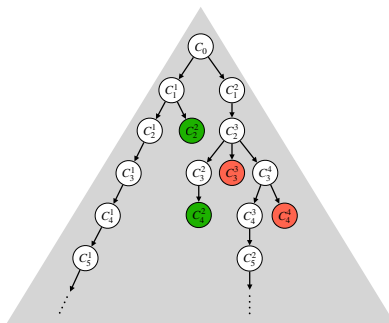
Definición:

- M **acepta** w si **existe** una ejecución ρ de M sobre w que cumple:
 - $\rho = C_0, \dots, C_m$ es finita. (M se detiene con w)
 - C_m es una configuración de aceptación.
 - Decimos que ρ es una **ejecución de aceptación** para M y w .
- El **lenguaje aceptado por** M se define como:

$$L(M) = \{w \in \Sigma^* \mid M \text{ acepta } w\}.$$

Recordatorio: visualizando MTNs

Las ejecuciones de M sobre w forman el **árbol de configuraciones** $\mathcal{T}_{M,w}$.



- Los nodos están etiquetados con configuraciones.
La raíz tiene la configuración inicial.
- Los hijos de un nodo con etiqueta C están etiquetados con las siguientes configuraciones de C .
- Las hojas están etiquetadas con configuraciones de detención (**aceptación** o **rechazo**).

Recordatorio: visualizando MTNs

Las ejecuciones de M sobre w forman el **árbol de configuraciones** $\mathcal{T}_{M,w}$.

M acepta $w \iff$ existe una **ejecución de aceptación** para M y w

$\iff$ existe una **rama** en $\mathcal{T}_{M,w}$

que termina en una **configuración de aceptación**

(**rama** = camino que comienza en la raíz y termina en algún nodo)

Una forma útil de pensar en una MTN M :

- Dada una entrada $w \in \Sigma^*$, comenzamos en la **configuración inicial**.
- Si M se encuentra con más de una opción de transición, escoge sólo una de las transiciones.
- El objetivo de M es “adivinar” una **ejecución de aceptación**.
- M tiene la habilidad especial de tener **mucha suerte**:
si existe una ejecución de aceptación, M adivina correctamente.

Usando MTNs

Las MTNs son muy útiles para chequear existencia de soluciones:

- Dada una instancia w , ¿existe una solución para w ?
- Con una MTN podemos adivinar una solución para w .
(en caso que exista.)

Ejemplo:

La siguiente MTN M acepta SAT:

Sobre entrada $\langle \varphi \rangle$:

1. Adivinar una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de una MTN

Sea una MTN N que para toda entrada, **todas** sus ejecuciones son finitas.

Definición:

- Sea una palabra $w \in \Sigma^*$ y $\rho = C_0, \dots, C_m$ una ejecución de M con w . Denotamos por $\text{largo}(\rho)$ al largo m de la ejecución ρ .
- Sea una palabra $w \in \Sigma^*$. Definimos el **tiempo de ejecución** $t_M(w)$ de M sobre w como:

$$t_M(w) = \max\{\text{largo}(\rho) \mid \rho \text{ ejecución de } M \text{ con } w\}.$$

- El **tiempo de ejecución** de M es la función $t_M : \mathbb{N} \rightarrow \mathbb{N}$ tal que:

$$t_M(n) = \max\{t_M(w) \mid w \in \Sigma^* \text{ y } |w| = n\}.$$

Comentarios:

- Decimos que M **toma tiempo** t_M .
- Si M toma tiempo $O(f(n))$ entonces:

Para toda palabra w de largo n y toda posible ejecución ρ de M con w , el largo de ρ es $\leq c \cdot f(n)$. (para $n \geq n_0$.)

Complejidad no-determinista de un lenguaje y NTIME

Definición:

Sea una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$.

- Un lenguaje L es **decidible en tiempo no-determinista** $O(f(n))$, si existe una MTN M que decide L tal que $t_M(n) = O(f(n))$.
- La clase de complejidad **NTIME** $(f(n))$ es la clase de todos los lenguajes L que son decidibles en tiempo no-determinista $O(f(n))$.

Comentarios:

- Si $f(n) = O(g(n))$, entonces $\text{NTIME}(f(n)) \subseteq \text{NTIME}(g(n))$.
- $\text{DTIME}(f(n)) \subseteq \text{NTIME}(f(n))$, para toda función f .

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Consecuencia:

- $L \in \text{NP} \iff L$ es decidable en tiempo no-determinista $O(n^k)$, para algún $k \in \mathbb{N}$.
- Decimos que L es decidable en tiempo polinomial no-determinista.

Ejemplos

Veamos que **SAT** es decidable en tiempo polinomial no-determinista.

Podemos tomar la siguiente **MTN** M :

Sobre entrada $\langle \varphi \rangle$:

1. Adivinar una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

Ejemplos

Veamos que **COMPOSITES** es decidable en tiempo polinomial no-determinista.

Podemos tomar la siguiente **MTN** M :

Sobre entrada $\langle n \rangle$:

1. Adivinar un entero $1 < p < n$.
2. Si p divide a n , **aceptar**. En caso contrario, **rechazar**.

Ejemplos

Veamos que 3-COL es decidable en tiempo polinomial no-determinista.

Podemos tomar la siguiente MTN M :

Sobre entrada $\langle G \rangle$:

1. Adivinar una función c de los nodos de G a $\{1, 2, 3\}$.
2. Si para toda arista (u, v) de G se cumple $c(u) \neq c(v)$, **aceptar**.
En caso contrario, **rechazar**.

Recordatorio: de MTNs a MTDs

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

Definimos:

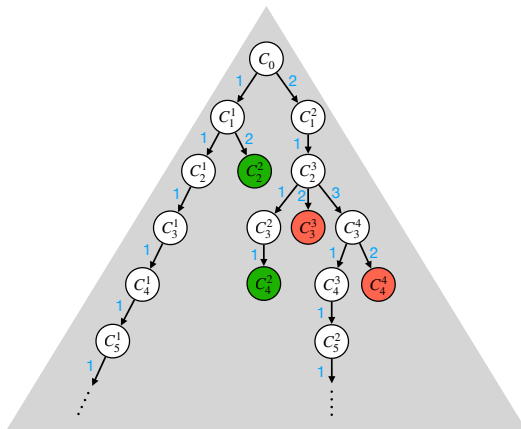
- $\Delta(q, a) = \{(q', b, D) \in Q \times \Gamma \times \{\triangleleft, \triangleright\} \mid (q, a, q', b, D) \in \Delta\}$.
- $m = \max\{|\Delta(q, a)| \mid q \in Q, a \in \Gamma\}$.

Enumeramos los triples de $\Delta(q, a)$, comenzando desde 1.

- Cada triple en $\Delta(q, a)$ recibe su propia **dirección** en $\{1, \dots, m\}$.

Cada **rama** de $\mathcal{T}_{M,w}$ se puede representar como una palabra en $\{1, \dots, m\}^*$.
(**secuencia de direcciones**)

Recordatorio: de MTNs a MTDs



- Ramas de aceptación: 12 y 2111
- Ramas de rechazo: 212 y 2132

Recordatorio: de MTNs a MTDs

Dado: $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta_M)$ una MTN.

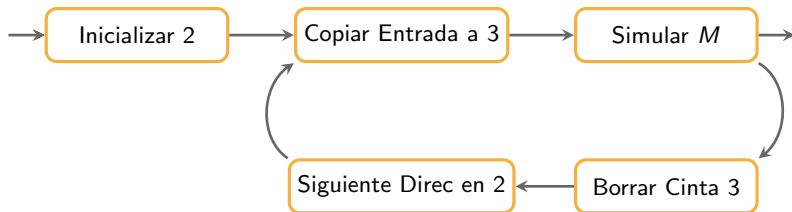
La MTD simuladora S tiene 3 cintas tal que:

- **Cinta 1:** mantiene la **palabra de entrada** w sin modificarla.
- **Cinta 2:** secuencia de **direcciones** que indica qué transiciones tomar.
- **Cinta 3:** **ejecuta** de M según las direcciones en Cinta 2.

Actualizamos la secuencia de direcciones de la **Cinta 2** según el orden lexicográfico $\leq$.

Recordatorio: de MTNs a MTDs

Estructura de la MTD S :



NP y no-determinismo

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Demostración ($\Rightarrow$):

Sea L tal que existe polinomio p y MTD polinomial M tal que para todo w :

$$w \in L \iff \text{existe } x \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } \langle w, x \rangle.$$

La siguiente MTN N decide L en tiempo polinomial:

Sobre entrada w :

1. Adivinar una palabra x con $|x| \leq p(|w|)$.
2. Ejecutar $M(w, x)$. Si M acepta, **aceptar**. En caso contrario, **rechazar**.

NP y no-determinismo

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Demostración ($\Leftarrow$):

Sea L decidible por una MTN N que toma tiempo polinomial $p: \mathbb{N} \rightarrow \mathbb{N}$.

Tomemos el siguiente verificador polinomial M :

Sobre entrada $\langle w, x \rangle$:

1. Chequea que x sea una secuencia de direcciones u con $|u| \leq p(|w|)$.
2. Ejecuta M con w , siguiendo las direcciones u .

Si M acepta, **aceptar**. En caso contrario, **rechazar**.

Se cumple para todo w :

$w \in L \iff$ existe una secuencia de direcciones u tal que M acepta w al seguir u .

$\iff$ existe x tal que $|x| \leq p(|w|)$ y M acepta $\langle w, x \rangle$.

NP y EXP

Observación: $P \subseteq NP$. (¿por qué?)

Teorema:

Sea $t : \mathbb{N} \rightarrow \mathbb{N}$. Toda MTN N que toma tiempo $t(n)$, puede ser simulada por una MTD con 3 cintas que toma tiempo $2^{O(t(n))}$.

Demostración:

Propuesta. (revisar la simulación de MTNs a MTDs.)

Consecuencia: $NP \subseteq EXP$.

Tenemos:

$$P \subseteq NP \subseteq EXP$$

(se sabe que $P \subsetneq EXP$.)

$\zeta P \neq NP?$

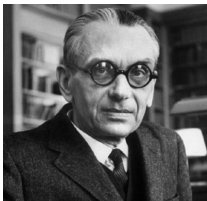
"If $P=NP$, then the world would be a profoundly different place than we usually assume it to be. There would be no special value in "creative leaps," no fundamental gap between solving a problem and recognizing the solution once it's found. Everyone who could appreciate a symphony would be Mozart; everyone who could follow a step-by-step argument would be Gauss..."

(Scott Aaronson, *Reasons to Believe*)

<https://scottaaronson.blog/?p=122>

"The main argument in favor of $P \neq NP$ is the total lack of fundamental progress in the area of exhaustive search. This is, in my opinion, a very weak argument. The space of algorithms is very large and we are only at the beginning of its exploration... ...The resolution of Fermat's Last Theorem also shows that very simple questions may be settled only by very deep theories."

(Moshe Y. Vardi)



Kurt Gödel (1906 - 1978)



John Von Neumann (1903 - 1957)

"... One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n , allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\Psi(F, n)$ be the number of steps the machine requires for this and let $\phi(n) = \max_F \Psi(F, n)$. The question is how fast $\phi(n)$ grows for an optimal machine..... If there really were a machine with $\phi(n) \sim k \cdot n$ (or even $\sim k \cdot n^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem. Now it seems to me, however, to be completely within the realm of possibility that $\phi(n)$ grows that slowly... "

(Carta pérdida de Gödel a Von Neumann, 1956)

<https://rjlipton.com/the-gdel-letter/>

$\zeta P \neq NP?$